

Sky-Spy — Complete Hardware Guide

Firmware: Sky-Spy (Drone RemoteID detector) · **Upstream:** [colonelpanichacks/Sky-Spy](#) (Apache-2.0, inherited from OpenDroneID; verify per-file headers) **Chips:** ESP32-S3 (8MB), ESP32-C6 (4MB) · **Cyber Controller profile:** sky-spy (esptool backend, merged single-bin @ 0x0, **detector-only — no transmit, no suicide**) **This guide:** purchase → build → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

Sky-Spy is a **passive drone Remote ID detector** for ESP32. It listens for the broadcast beacons that modern drones (UAS) are required to emit under **FAA Remote ID** and the **ASTM F3411 / Open Drone ID** standard, then surfaces the operator and aircraft data those beacons carry — drone GPS position, altitude, the pilot/operator (ground-station) location, and the drone's Basic ID. It does this two ways at once:

- **WiFi:** promiscuous-mode capture on **channel 6** (the WiFi RemoteID standard channel), parsing Open Drone ID frames carried in WiFi beacons/NAN action frames.
- **BLE:** active scanning (≈ 100 ms interval, ~ 1 s scan windows) for **ASTM F3411 RemoteID advertisements**, keyed on **Service UUID 0xFFFA**.

Detections are emitted as **line-delimited JSON over USB serial at 115200 baud**, and an on-board **passive buzzer** gives audible alerts. On a dual-core S3 the work is split (one core for WiFi, one for BLE/output) so neither path starves the other. This is **detection and situational awareness only** — Sky-Spy never transmits, jams, or spoofs. Cyber Controller's role is to flash the firmware and to connect to the device so you can watch / log the detection stream. *Note: Sky-Spy detects drone Remote ID broadcasts — it is not an ADS-B aircraft receiver and does not track crewed aviation.*

2. Legal & Safety

Sky-Spy is receive-only and lawful to operate in most places, but know your local rules. - Detector, not a weapon. It only *listens* to beacons drones already broadcast publicly. It does **not** jam, spoof, take over, or interfere with any aircraft. Jamming or spoofing drone control/Remote ID is a serious federal offense in the US (FAA/FCC) and illegal in most jurisdictions — Sky-Spy does none of that. - **Intended use:** lawful public-safety, airspace awareness, event/perimeter monitoring, research, and understanding the Remote ID ecosystem on spectrum you are allowed to receive. - **Receiving vs. recording.** Passively receiving public RF beacons is broadly permitted, but **logging, retaining, or acting on** operator/pilot location data can implicate privacy, surveillance, and data-protection law (varies by country/state). Don't use detections to harass or interfere with lawful operators. **verify:** local RF-reception and privacy law before deploying or retaining logs. - Remote ID **mandates and exemptions differ by region** — not every drone you see will broadcast, and a "no detection" does not mean "no drone."

3. Hardware & Purchasing

Sky-Spy targets two chip families. The **ESP32-S3** build is the reference target; the **ESP32-C6** is the WiFi-6 alternative. Pick by board:

Tier	Board	Chip / specs	Why	Where to buy (search terms)
Official / best	OUI-SPY board (Colonel Panic)	ESP32-S3, integrated PWM buzzer, built-in + external (u.FL) antenna, USB-C	Purpose-built for this firmware; no wiring	colonelpanic.tech (maker moved off Tindie). verify current price/stock — was ~US\$75 assembled / ~US\$20 bare PCB
Reference DIY	Seeed Studio XIAO ESP32-S3	ESP32-S3 dual-core LX7 @240 MHz, 8MB flash, 8MB PSRAM, BLE 5.0, u.FL antenna	Primary supported board; cheap, tiny	seeedstudio.com "XIAO ESP32-S3"; Amazon "Seeed Studio XIAO ESP32S3"
WiFi-6 DIY	Seeed Studio XIAO ESP32-C6	ESP32-C6 RISC-V, WiFi 6 + BLE 5.3, 4MB flash	Alternative target; newer radio	seeedstudio.com "XIAO ESP32-C6"; Amazon "Seeed Studio XIAO ESP32C6 (Pre-Soldered)"

Accessories: - A **USB-C data cable** (not charge-only) — XIAO boards and the OUI-SPY are USB-C. - A **passive piezo buzzer** if you build on a bare XIAO and want audible alerts (the OUI-SPY has one on-board). Wire **buzzer +** → **GPIO3 (silkscreen "D2")**, **buzzer -** → **GND**, optional **~100 Ω** in series to tame volume. **verify:** the buzzer GPIO against the README for your firmware revision/board. - Optional **u.FL** → **SMA pigtail + 2.4 GHz antenna** for better range on boards that expose u.FL. - The XIAO ESP32-S3 / C6 **antenna must be attached** to its u.FL connector for usable sensitivity.

Buy genuine Seeed XIAO boards (clones may ship wrong flash size or a missing PSRAM part — the S3 build expects 8MB flash / 8MB PSRAM).

4. Building / Assembly

Sky-Spy is built with **PlatformIO** (there are no published GitHub release binaries at time of writing — see §5). Two PlatformIO environments exist:

- `seeed_xiao_esp32s3` (recommended)
- `seeed_xiao_esp32c6`

Typical workflow (from the cloned repo):

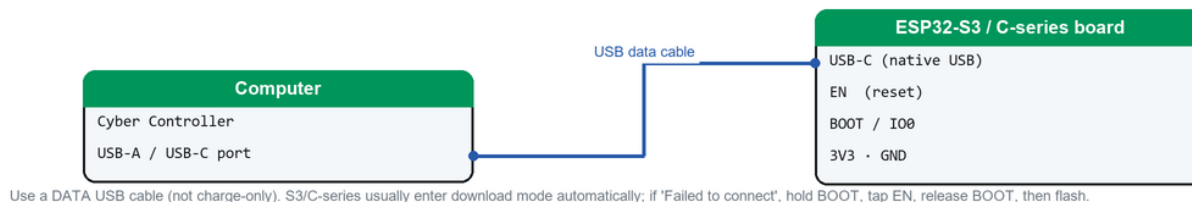
```
pio run -e seeed_xiao_esp32s3          # build only
pio run -e seeed_xiao_esp32s3 -t upload # build + flash over USB
pio device monitor -e seeed_xiao_esp32s3 # open the 115200 serial monitor
```

Swap `-e seeed_xiao_esp32c6` for the C6 board. `pio run -e <env> -t clean` forces a clean rebuild.

- **Toolchain/deps** (PlatformIO resolves automatically): ESP32 Arduino Core **v3.2.0** (ESP-IDF v5.4), **ArduinoJson v6.21.5**, ESP32 BLE Arduino. **verify:** exact versions in `platformio.ini` for your checkout.
- **Drivers:** XIAO ESP32-S3/C6 and OUI-SPY use the chip's **native USB-CDC** — no CP210x/CH340 driver needed; the COM/tty appears when plugged in. (If a board variant routes through a USB-UART bridge, install that bridge's driver.)
- **Pre-built OUI-SPY:** no assembly — plug in a data cable.
- **Bare XIAO build:** the only hardware step is the optional buzzer on GPIO3 (see §3); the radio uses the on-module antenna.

5. Flashing & First Run (via Cyber Controller)

Flashing connection — ESP32 (native USB)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

Sky-Spy flashes as a **single merged .bin written at offset 0x0** (esptool backend) — simpler than multi-file firmware. In Cyber Controller:

1. Connect the board by USB; open Cyber Controller → **Flash** tab.
2. **Port:** select the board's COM/tty (click *Refresh* if missing — replug / check the data cable).
3. **Firmware Profile:** Sky-Spy (profile sky-spy).
4. **Board / chip:** pick **ESP32-S3 Generic** (8MB) or **ESP32-C6 Generic** (4MB) to match your board. Both use `flash_mode = dio`, `flash_freq = 80m`.
5. Click **Flash**. Cyber Controller resolves the firmware via the GitHub-release resolver (latest .bin asset, matched per chip), then writes the merged image at 0x0 at the default **115200** baud.

Important — firmware availability. The upstream repo currently has **no published GitHub releases**, so the release resolver may return nothing (it degrades to "source only / empty" rather than erroring). If the Flash tab shows no asset: - **verify** the Releases page (<https://github.com/colonelpanichacks/Sky-Spy/releases>) for a newly published .bin, **or** - build the .bin yourself with PlatformIO (§4) and flash that artifact (esptool, merged @ 0x0), **or** - check whether you actually want the unified OUI-SPY firmware build (see §6 note).

First boot: the device starts both scanners immediately, prints a startup banner, and begins emitting JSON when a drone is seen. The buzzer's periodic **double beep (~600 Hz every ~5 s)** is the heartbeat that confirms it's alive; **three quick ~1000 Hz beeps** signal a detection.

6. Integrate into Cyber Controller

- **Profile:** sky-spy (src/config/profiles/sky_spy.json) — backend **esptool**, image_model: merged-single-bin, app_offset: 0x0, resolver github_release (asset regex ^(.*)\.bin\$, chip mapped from the asset name). supports_suicide: false (nothing to self-destruct — it never transmits).
- **Boards defined:** esp32s3 (8MB) and esp32c6 (4MB), both dio/80m, default baud **115200**.
- **Control / monitoring:** open the **Devices** tab → select the port → **Connect**. Because Sky-Spy's protocol is null, Cyber Controller treats it as a **plain serial detector**: the persistent terminal shows the raw JSON detection stream (and the 60-second status line). There is **no command palette / attack verbs / Targets-pool parser** for this profile — that's by design; Sky-Spy is a one-way detector, not an interactive offensive tool like Marauder.
- **Logging:** capture the terminal output to a file for later mapping/analysis (lawful retention only — §2).
- **Backup first:** use **Backup** in the Flash tab to dump existing flash before overwriting (useful if the board previously ran a different firmware).
- **Note — unified firmware:** Colonel Panic also ships a combined OUI-SPY image ("detector + flockyou + foxhunter + skyspy in one build"). That is a *different* firmware/profile than this standalone Sky-Spy build; **verify** which one you intend to flash. This guide and the sky-spy profile cover the standalone Sky-Spy detector.

7. Usage (end-to-end)

1. **Power on / connect.** Plug in the board; in Cyber Controller's Devices tab, **Connect** at 115200. The heartbeat double-beep confirms it's scanning.
2. **Wait for detections.** When a Remote-ID-broadcasting drone is in range, Sky-Spy beeps (3× ~1000 Hz) and emits a JSON line, e.g.: `json`

```
{"mac": "aa:bb:cc:dd:ee:ff", "rssi": -45, "drone_lat": 37.7749, "drone_long": -122.4194, "drone_altitude": 120, "pilot_lat": 37.7750, "pilot_long": -122.4195, "basic_id": "1234567890ABCDEF"}
```

Fields: `mac`, `rssi` (signal strength → rough proximity), `drone_lat/long`, `drone_altitude`, `pilot_lat/long` (operator/ground-station), `basic_id` (Open Drone ID identifier).
3. **Read proximity by RSSI.** A rising RSSI (less negative) means the drone is getting closer; use it for rough direction-finding by moving and watching the value.
4. **Map it (optional).** The upstream `mesh-mapper.py` tool consumes the serial JSON and plots drones + operators on a live map: `bash python mesh-mapper.py --port /dev/cu.usbmodem1101 # use your actual port` (Close Cyber Controller's connection first — only one app can own the serial port at a time.) A related **drone-mesh-mapper** project relays detections over **Meshtastic** for off-grid mesh alerting.
5. **Both bands run continuously** — no mode switching needed; WiFi (ch 6) and BLE (UUID `0xFFFA`) scan in parallel. Disconnect in Cyber Controller to free the port when done.

8. Troubleshooting

- **No COM/tty port:** use a **data** USB-C cable (not charge-only); replug; the XIAO/OUI-SPY use native USB-CDC so no driver is normally needed. On Linux add your user to `dialout` and replug.
- **Flash tab shows no firmware / nothing to download:** upstream has no GitHub release yet — build the `.bin` with PlatformIO and flash that, or **verify** the Releases page for a new asset (see §5).
- **"Failed to connect" while flashing:** put the board in **download mode** — on XIAO ESP32-S3/C6 hold **BOOT**, tap **RESET**, release BOOT (or hold BOOT while plugging in); lower the flash baud in Settings; close any serial monitor (Cyber Controller terminal or `pio device monitor`) holding the port.
- **No detections at all:** confirm a Remote-ID-broadcasting drone is actually in range (older/exempt drones may not broadcast); ensure the **antenna is attached** (u.FL); remember WiFi capture is **channel 6** — a drone advertising only on BLE will still be caught by the BLE path, and vice-versa.
- **No buzzer / no beeps:** on a bare XIAO the buzzer is optional — check wiring (+ → **GPIO3/D2**, – → **GND**), use a **passive** piezo, and **verify** the buzzer GPIO in the README for your revision.
- **Garbled serial / wrong data:** set the monitor to **115200** baud; only one program may hold the port.
- **Boot-loop after flashing:** confirm the image matches the chip (S3 vs C6) and the board's flash size (S3 = 8MB, C6 = 4MB); use **Erase Flash** then re-flash.
- **C6 vs S3 mismatch:** flashing an S3 build to a C6 (or vice-versa) won't run — pick the matching board in the Flash tab.

9. Sources

- **Upstream firmware:** colonelpanichacks/Sky-Spy — <https://github.com/colonelpanichacks/Sky-Spy> (README: boards, PlatformIO envs `seeed_xiao_esp32s3`/`seeed_xiao_esp32c6`, GPIO3 buzzer, channel 6 / BLE UUID `0xFFFA`, JSON serial format, dependency versions). Releases checked: none published at time of writing.
- **Mapping/mesh tooling:** `mesh-mapper.py` (in-repo) and colonelpanichacks/drone-mesh-mapper (Meshtastic).
- **Standards:** ASTM F3411 / Open Drone ID; FAA Remote ID rule (background on what is being detected).
- **Cyber Controller profile:** `src/config/profiles/sky_spy.json` (id `sky-spy`, `esptool`, `merged-single-bin @ 0x0`, boards `esp32s3 8MB` / `esp32c6 4MB`, baud `115200`, `supports_suicide:false`, release resolver).
- **Hardware/purchasing:** OUI-SPY board — colonelpanic.tech (maker moved off Tindie; **verify** price/stock); Seeed Studio XIAO ESP32-S3 ([seeedstudio.com p-5627](https://www.seeedstudio.com/p-5627)) and XIAO ESP32-C6 ([seeedstudio.com p-5884](https://www.seeedstudio.com/p-5884)); also on Amazon (Seeed Studio store).
- Vendor links, prices, and firmware-release availability change — **verify at purchase/flash time.**